Call Manager — Project Documentation

1. Executive Summary

Call Manager is an AI-powered outbound calling platform built for **The Shri Ram Academy (TSRA)**, an IB day-boarding school in Gachibowli, Hyderabad. It automates admissions outreach: an AI voice agent ("Arjun") calls prospective families, answers questions about the school using live-scraped website content, books campus visit appointments, schedules callbacks for people who can't talk right now, and syncs everything to Google Calendar and email — all without a human dialer.

Live deployment:

- Frontend: `https://f1zc6ndiau3myh45t5o2e088.datalabscorp.ai`
- Backend API: `https://tl9egburoq7ikdxfezshedz5.datalabscorp.ai`
- Hosted on **Coolify** (self-hosted PaaS), branch `coolify_deployment`

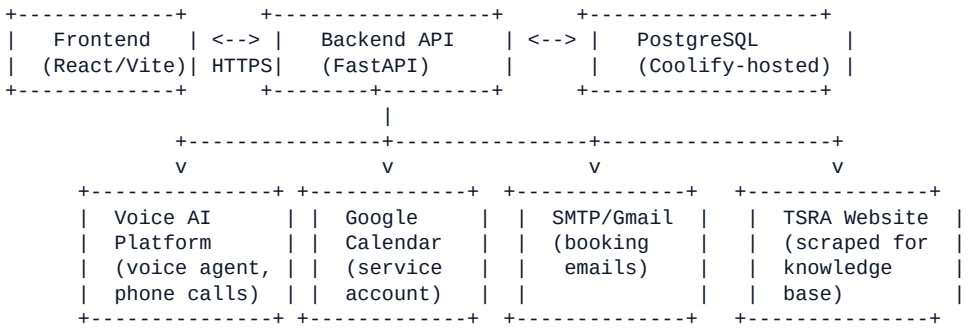
---

2. What the System Does

1. **Admin uploads a lead list** (Excel/CSV) through the dashboard — this becomes a "Campaign."
2. **The system auto-dials** contacts in that campaign, respecting a configurable concurrency limit, using a **third-party voice AI platform** to place real phone calls with a conversational voice agent.
3. **The AI agent ("Arjun")** follows a strict, single-prompt-driven script:
    - Opens with a fixed identity check ("Hi, am I speaking with {name}?")
    - Answers factual questions about the school (fees, curriculum, admissions process, facilities) by querying a **live knowledge base** scraped from the school's website — never from memory, since content can change
    - Detects reschedule requests ("call me later") and books a follow-up call automatically
    - Books campus visit appointments, collecting/confirming the caller's email
    - Detects "not interested"/do-not-call requests and stops immediately, no re-pitching
    - Records a structured outcome at the end of every call
4. **On appointment booking**, the system (in the background, with zero added latency to the call):
    - Creates a calendar event on the school's Google Calendar
    - Sends the caller a styled HTML confirmation email
5. **On a reschedule request**, the system automatically re-dials the contact at the agreed time via a background scheduler — no manual follow-up needed.
6. **Admins/staff** manage everything from a web dashboard: campaign status, per-contact call history (transcripts + audio recordings), scheduled callbacks, booked appointments, and system settings.

---

3. Architecture



- **Frontend** (React 19 + TypeScript + Vite) — 4 pages: Campaigns, Leads Directory, Scheduling, System Settings. Served via Nginx in production.
- **Backend** (FastAPI + SQLAlchemy) — REST API, background job scheduler (APScheduler), and webhook receivers for the voice AI platform and Cal.com events.
- **Voice agent** — a single shared voice AI agent (not duplicated per campaign) driven entirely by one prompt file (`agent_prompt.md`), with 5 custom tools it can call mid-conversation.
- **Database** — PostgreSQL in production (Coolify-managed), with SQLite as an intentional local-dev-only fallback (no silent fallback in production — a misconfigured connection now fails loudly at startup rather than masking data loss).

---

4. Tech Stack

Layer	Technology
Backend framework	FastAPI (Python 3.10)
ORM / DB	SQLAlchemy, PostgreSQL (prod) / SQLite (local dev)
Auth	JWT (python-jose), bcrypt password hashing
Job scheduling	APScheduler
Voice AI	Third-party voice AI platform SDK, voice: ElevenLabs "Adrian"
Calendar	Google Calendar API via a service account (`google-api-python-client`)
Email	SMTP (Gmail)
Knowledge base	BeautifulSoup4 web scraping of the school website
Frontend framework	React 19 + TypeScript + Vite

Layer	Technology
Frontend styling	Custom CSS design system (glassmorphism, CSS variables, dark/light themes)
Deployment	Docker + Docker Compose, hosted on Coolify
Dev tunnel (local only)	ngrok

---

5. Core Workflows

5.1 Outbound Campaign Calling

1. Admin uploads Excel/CSV → contacts created, campaign ("UploadBatch") created.
2. Admin clicks "Start" → the dialer (`dialer.py`) fires calls up to a configurable concurrency limit, per-campaign (independent queues so one campaign's pace doesn't block another).
3. As each call ends, the dialer automatically fires the next queued contact in that same campaign.
4. The voice AI platform sends `call_started` / `call_ended` / `call_analyzed` webhooks back to the backend, which records outcome, transcript, recording, and duration, and updates contact status.

5.2 Reschedule / Callback Flow

1. Caller says they can't talk now → the agent calls the `schedule_callback` tool with a resolved absolute date/time.
2. A background scheduler job fires the callback automatically at that exact time.
3. **Retry logic**: if a call goes unanswered, or the caller hangs up mid-conversation with nothing resolved, the system automatically reschedules — capped at 2 automatic retries (3 total attempts) before giving up and flagging for manual follow-up.

5.3 Appointment Booking

1. Caller wants a campus visit → agent collects/confirms date, time, and email, then calls `book_appointment`.
2. The appointment is created instantly (no added latency on the live call); calendar sync and email dispatch happen in the background.
3. Rebooking the same contact updates the existing appointment in place rather than creating a duplicate.
4. **Note**: the calendar event does not include the caller as a formal Google Calendar "attendee" — service accounts without Google Workspace domain-wide delegation are blocked by Google from doing this. Their contact info goes in the event description instead; they're still notified via the confirmation email.

5.4 Knowledge Base

- The school's website is scraped periodically (and on-demand) into `KnowledgeChunk` records.

- Every factual question the agent answers is grounded in this live-scraped content via a `lookup_school_info` tool call — never from the LLM's own memory — so answers stay accurate even if the school updates its website.

---

6. Voice Agent Details

- **Persona**: "Arjun," warm/professional admissions assistant — defined entirely in `backend/agent_prompt.md`, the single source of truth (no per-campaign forks).
- **Voice**: ElevenLabs `11labs-Adrian`.
- **Tools available to the agent**: `lookup_school_info`, `schedule_callback`, `book_appointment`, `mark_outcome`, `end_call`.
- **Hard behavioral rules baked into the prompt**: never invent school facts, always resolve relative times against the actual call timestamp (IST), always confirm before booking, detect reschedule/do-not-call intent from many phrasings, respond in English regardless of the caller's language, and always record an outcome before ending every call.
- A setup script manages the agent's lifecycle: first run creates it, every subsequent run patches the same agent's prompt/tools/voice/webhook URL in place — this script must be re-run any time the prompt, voice, or webhook base URL changes, or the live agent won't pick up the change.

---

7. Database Schema (7 tables)

Table	Purpose
`contacts`	Leads uploaded per campaign — name, phone, email, status
`upload_batches`	Campaigns (one per uploaded file)
`call_attempts`	Every call made — outcome, duration, transcript, recording URL
`scheduled_callbacks`	Pending/fired reschedule requests
`appointments`	Booked campus visits, with Google Calendar links
`knowledge_chunks`	Scraped website content for the agent's factual answers
`settings`	Runtime-configurable API keys/credentials (masked in the UI when secret)

---

8. Security Posture

- Voice AI platform and Cal.com webhooks are **signature-verified** (HMAC) when a real API key/secret is configured.
- The voice AI platform's custom-function tool endpoints (`book_appointment`, `schedule_callback`, etc.) support an optional shared-secret header (`AEGIS_TOOLS_SECRET`) — **recommended but not yet enabled** in the current deployment (these endpoints are currently unauthenticated and publicly reachable; anyone who discovers the URL could invoke them directly).
- Knowledge base admin endpoints require authentication.
- JWT-based session auth for the dashboard, with role separation (admin vs staff).
- Demo login credentials and JWT secret are still at their default values in the current environment — **should be replaced with real values before wider use.**

---

9. Deployment / Infrastructure

- **Platform**: Coolify (self-hosted PaaS) at `coolify_deployment` branch.
- **Two services**, each its own Docker container: `backend` (FastAPI, port 5000 internally) and `frontend` (Nginx serving the Vite build, port 80 internally — Coolify's reverse proxy handles the public domain/TLS).
- **Database**: PostgreSQL, hosted as a separate Coolify-managed service; the backend connects over Coolify's internal Docker network.
- Frontend's backend API URL (`VITE_API_URL`) is a **build-time** variable — must be set in Coolify's environment config for the frontend service and requires a full rebuild (not just restart) to take effect.
- No ngrok/tunnel is used in production — that's a local-development-only mechanism for exposing a laptop to the voice AI platform's webhooks during testing.

---

10. Known Limitations / Recommended Next Steps

1. **Tool webhook authentication** (`AEGIS_TOOLS_SECRET`) is not yet enabled in production — recommended before scaling up call volume.
2. **Google Calendar attendee invites** are not possible with the current service-account setup (Google platform limitation, not fixable without upgrading to a paid Google Workspace domain with delegated authority). Callers are still notified via email.
3. **Auto-retry logic** for unanswered/incomplete calls is capped at 2 retries; no cap currently exists on caller-initiated reschedule requests ("call me back later" can be requested indefinitely).
4. **Default demo credentials/JWT secret** should be rotated before this is used with real, sensitive lead data at scale.
5. Two duplicate legacy test contacts exist from early testing (same phone number, near-identical emails) — harmless but worth cleaning up.

Document generated from a full review of the live codebase, configuration, and deployment as of 2026-07-07.